AetherRuntime: A Falsification-First Investigation of Runtime Assurance in the Aether Family

A Falsification-First Investigation of Monitoring, Detection, Containment, and Recovery in Multi-Agent Software Ecosystems

Samir Baladi

Interdisciplinary AI Research — Ronin Institute / Rite of Renaissance

Research Article — Distributed Systems / Software Engineering / Institutional Analysis

DOI: https://doi.org/10.5281/zenodo.24078477

ORCID: 0009-0003-8903-0029 · gitdeeper@gmail.com

Version 0.1.0 — November 2026

Status: *Architectural Hypothesis Under Investigation (v0.1.0). This is not a committed project. It is a structured investigation of whether a runtime-assurance gap genuinely exists in the Aether family. No implementation is claimed. No real-world deployment has been evaluated. The final project structure (single, dual, triple, or quadruple) is undetermined.*

Abstract

The Aether family comprises five preprints that address how agent-generated changes are stored (AetherGit), integrated and verified operationally (AetherFlow), specified and formally evidenced (AetherProof), synchronized without a central coordinator (AetherMesh), and authorized under policy (AetherGovern). None is primarily concerned with what agents do while they run: monitoring, detection of anomalous behaviour, containment, halting, and recovery. This paper investigates whether that omission is a genuine gap that warrants a sixth functional dimension, tentatively AetherRuntime, or an artifact of how the existing dimensions were scoped. It uses a falsification-first method: five failure conditions (F1–F5) and a decision rule are stated before the candidate is described, and 'not falsified' is never promoted to 'confirmed' on the basis of argument. The paper compares three candidate capabilities, specifies a five-layer runtime architecture (monitoring, detection, containment, halt, recovery), and states two propositions with proof sketches. The first concerns determinism: local anomaly detection can coexist with AetherMesh's requirement that admission depend only on a delta and its causal closure, provided detection verdicts shape local actions and signed observation records but not the admitted set. The second concerns timing: any decentralized halt protocol leaves a window between detection and enforcement that cannot be eliminated and is at least the sum of dissemination, queueing, and verification delays. A resource-consumption metric is given a dimensionally consistent, windowed form. By argument alone, F1, F2, and F4 are not falsified in narrowed form, F3 (boundary with governance) is the most fragile condition, and F5 (observable benefit) is untested. AetherRuntime is a candidate, not a result. No implementation, experiment, or measurement is reported.

Keywords: *Runtime Assurance; Anomaly Detection; Containment; Halt Protocols; Recovery; Multi-Agent Systems; Falsification; Decentralized Systems; Agentic Software Engineering; AetherRuntime.*

Evidentiary status: *hypothesis under investigation. Every statement in this paper is labeled as a definition, a hypothesis, or an observation. Propositions P1–P2 remain proof sketches, not machine-checked. No empirical result is reported. Descriptions of third-party work follow the cited sources and were not independently verified.*

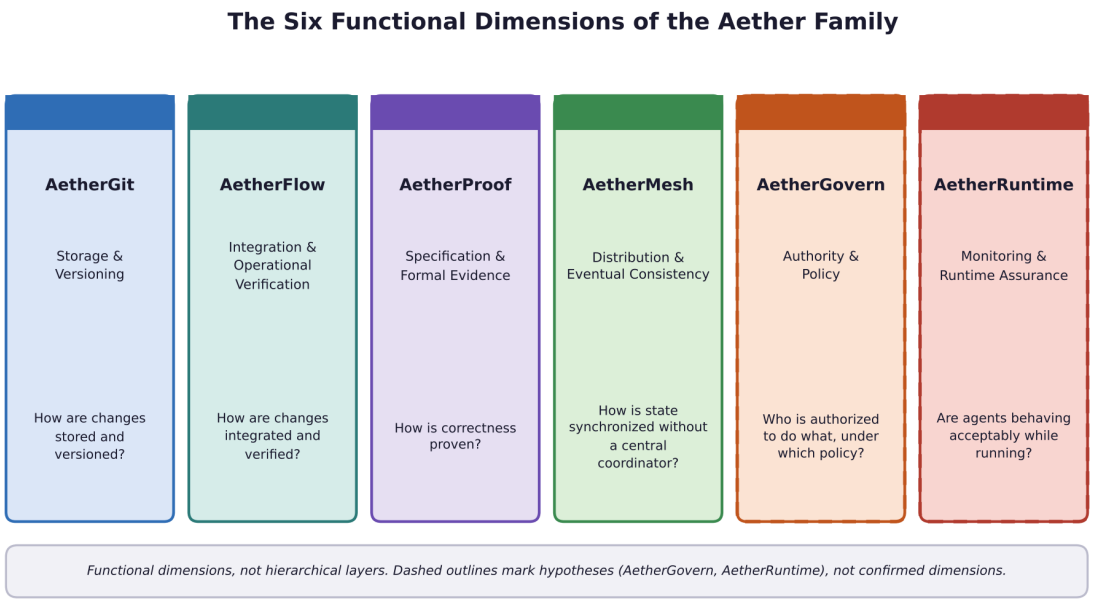


Figure 1 (Fig1_Six_Dimensions.png). The six functional dimensions of the Aether family. The fifth (AetherGovern) and sixth (AetherRuntime) dimensions are hypotheses under investigation, not confirmed dimensions. The dimensions are functional, not hierarchical; the figure was redrawn for this record.

Table of Contents

1. Introduction	3
2. Background and Related Work	5
3. The Falsification-First Methodology	8
4. The Three Candidate Capabilities	10
5. AetherRuntime: Architecture	11
6. AetherRuntime: Interfaces	21
7. Evaluation	22
8. Conclusion and Future Work	24
Appendix A — Notation	26
Appendix B — Failure Conditions in Detail	26
Appendix C — Open Questions Q1–Q20	26
Appendix D — Glossary	26
References	27
Research Status Statement	28

1. Introduction

1.1 The Coordination Bottleneck in Multi-Agent Ecosystems

Autonomous software agents now propose, revise, and merge code with little per-change human supervision. When many such agents share a codebase, the mechanisms that coordinate them become a limiting resource. Five Aether preprints (Baladi, 2026a–e) address successive aspects of the problem: how changes are stored and versioned, how they are integrated and verified, how correctness is specified and evidenced, how state is synchronized without a central coordinator, and how authority is granted and withdrawn. Each of those concerns is about *artifacts and decisions*: changesets, merges, specifications, replicas, and authorizations.

A different concern arises from the fact that agents are *running processes*. An agent can consume resources without bound, enter a loop, emit a burst of network traffic, drift from its usual behaviour, or be taken over by an attacker, all while every individual change it submits passes verification and every action it takes is authorized. Whether the family already has an adequate answer to this concern, or whether it forms a separable one, is the object of this investigation. The question is deliberately narrow and falsifiable: it concerns the structure of a family of architecture proposals, not the general importance of monitoring.

1.2 The Aether Family: Five Dimensions and a Candidate Sixth

Each of the five existing works is a preprint published on Zenodo. None has been peer reviewed, and, as far as the author's records state, none has an evaluated implementation. The fifth, AetherGovern, is itself framed as a hypothesis under investigation whose observable-benefit condition is untested; we therefore treat the five as *existing preprints* and not as five confirmed dimensions. This matters for Section 5, because the boundary between runtime assurance and governance (F3) is only as firm as governance itself.

Dimension	Focus	Question	Record
AetherGit	Storage and versioning	How are changes stored and versioned?	Baladi (2026a), 10.5281/zenodo.23966896
AetherFlow	Integration and operational verification	How are changes integrated and verified?	Baladi (2026b), 10.5281/zenodo.23979129
AetherProof	Specification and formal evidence	How is correctness proven?	Baladi (2026c), 10.5281/zenodo.24013124
AetherMesh	Distribution and eventual consistency	How is state synchronized without a central coordinator?	Baladi (2026d), 10.5281/zenodo.24038038
AetherGovern (hypothesis)	Authority and policy	Who is authorized to do what, under which policy?	Baladi (2026e), 10.5281/zenodo.24054524
AetherRuntime (hypothesis)	Monitoring and runtime assurance	Are agents behaving acceptably while running?	This paper, 10.5281/zenodo.24078477

As in the governance investigation, we use the word *dimension* and not *layer*. The relationships among the works are not strictly hierarchical, and a candidate sixth dimension must be shown to be *functionally distinct*, not merely to occupy a further position in a stack. Figure 1 shows the six dimensions.

1.3 The Central Question

Central question *(definition).* Is there a body of monitoring, detection, containment, halt, and recovery requirements for multi-agent software ecosystems that (i) is not already specified by the five existing dimensions, (ii) is not reducible to verification or to governance, (iii) can be specified without reintroducing a central authority, and (iv) yields an observable benefit over existing mechanisms?

The four clauses correspond to the failure conditions F1 (clause i), F2 and F3 (clause ii), F4 (clause iii), and F5 (clause iv), stated in Section 3. A negative answer to any clause would show that runtime assurance, as a separate dimension, is not warranted. A positive answer to all four, supported by evidence, would show that it is a candidate. **This is a candidate, not a result.**

1.4 Contributions to the Investigation

The following are contributions to an *investigation*; they are not committed architectural contributions and none is validated by implementation.

1. **A falsification-first procedure** of five failure conditions and a decision rule, stated before the candidate is described (Section 3, Appendix B).
2. **A comparison of three candidate capabilities** (orchestration, governance, runtime assurance), with the status of each made explicit (Section 4).
3. **A hypothesized five-layer runtime architecture** (monitoring, detection, containment, halt, recovery), including a state machine and a halt protocol (Section 5).
4. **Two propositions with proof sketches**: local anomaly detection can coexist with AetherMesh's determinism requirement if detection verdicts do not gate admission (P1); and any decentralized halt leaves an irreducible window (P2) (Section 5.7).
5. **A corrected, dimensionally consistent formulation of a resource-consumption metric**, with the defects of the naive form identified (Section 5.3).
6. **A provisional assessment of F1–F4 by argument** and an evaluation design that fixes failure criteria in advance (Sections 5.8 and 7), with twenty open questions.

1.5 Evidentiary Status

Every statement in this paper is a *definition* (it fixes a term), a *hypothesis* (plausible, not established), or an *observation* (a claim about the literature or the existing Aether records that a reader can check against the cited source). No fourth category of *result* is used because none is reported. **No real-world deployment has been evaluated. Propositions P1–P2 remain proof sketches, not machine-checked.**

Item	Status
Existence of a runtime-assurance gap	Hypothesis under investigation
Failure conditions F1–F5	Definitions (proposed test criteria)
Five-layer architecture, state machine, halt protocol	Hypothesis; specification only
Propositions P1–P2	Proof sketches; not machine-checked
Assessment of F1–F4	Argument only; provisional
Assessment of F5	Untested
Implementation, deployment, measurement	None
Project structure (single, dual, triple, or quadruple)	Undetermined

DOI: https://doi.org/10.5281/zenodo.24078477

1.6 Why Runtime Assurance?

Three considerations motivate looking at runtime assurance, and three considerations count against it. We list both so that the motivation is not mistaken for evidence.

For. *(Hypothesis.)* First, verification, in the sense of AetherProof and AetherFlow, evaluates changes and merged states; it does not constrain a process between changes. A runaway loop in an agent that is committing nothing is invisible to merge-time checks. Second, authorization, in the sense of AetherGovern, decides whether an action is permitted; an authorized agent can still behave abnormally within its permissions. Third, in a decentralized system, no single node observes every agent, so the question of who observes, who decides to act, and how a collective decision is reached has no default answer.

Against. *(Hypothesis.)* First, operating-system and container facilities already provide resource accounting and limits (Menage, 2007; Linux Kernel Documentation, 2024), and intrusion-detection research already provides anomaly-detection techniques (Denning, 1987; Axelsson, 2000; Chandola et al., 2009); a runtime dimension may be an integration of existing practice. Second, runtime monitoring of formally specified properties is a well-developed form of verification (Bartocci et al., 2018), so part of the candidate may belong to AetherProof. Third, a halt order is a withdrawal of authority, so part of the candidate may belong to AetherGovern. Each counter-argument corresponds to a failure condition.

Limitations of this section. This section states motivation, not evidence. The three counter-arguments may, singly or together, remove most of the candidate's content; whether they do is what Sections 5 and 7 examine.

2. Background and Related Work

This section reviews the Aether family, the runtime-assurance literature, and the boundaries with verification, governance, and operational verification. Descriptions of third-party work follow the cited sources and were not independently verified.

2.1 The Aether Family

AetherGit (Baladi, 2026a) proposes an append-only changeset log with parent pointers. **AetherFlow** (Baladi, 2026b) proposes post-merge verification and dynamic conflict resolution through a message-based protocol; its *operational verification* concerns checking integrated changes in operation, not monitoring a running agent process. **AetherProof** (Baladi, 2026c) proposes specification-driven proof construction and a human review protocol. **AetherMesh** (Baladi, 2026d) proposes decentralized synchronization from CRDTs (Shapiro et al., 2011; Almeida et al., 2018), epidemic gossip (Demers et al., 1987; Karp et al., 2000; Jelasity et al., 2005), and a lightweight Byzantine admission layer. **AetherGovern** (Baladi, 2026e) investigates authority, policy, accountability, revocation, and dispute resolution.

Five observations about these records bear directly on the investigation. *(Observation 1.)* AetherMesh requires admission predicates to be deterministic functions of a delta and its causal closure (its assumption A6), because otherwise honest nodes can admit different sets and diverge. *(Observation 2.)* AetherMesh treats rate limits and resource exhaustion by *deltas* as local delivery policies that delay but do not permanently reject (its threat T9); it does not address resource consumption by agent *processes*. *(Observation 3.)* AetherGovern proposes revocation of authority with an irreducible exposure window and a causal-authority admission rule (its Propositions 1 and 2); a halt order is, in part, a revocation. *(Observation 4.)* AetherGit's append-only log supports replay and reconciliation but, being append-only, cannot delete admitted records. *(Observation 5.)* AetherProof's trusted-computing-base manifests concern verification tools, not runtime monitors. Together, these observations suggest that fragments relevant to runtime assurance exist in the family and that the candidate's content is concentrated in what

is not covered: process-level monitoring and detection, graduated containment, and recovery.

2.2 Runtime Assurance Literature

Dependability. Avizienis et al. (2004) give a taxonomy of dependable and secure computing in which faults cause errors and errors cause failures, and in which fault tolerance comprises error detection, recovery (including rollback and rollforward), and fault handling. The proposed layers map onto this vocabulary: detection corresponds to error detection, containment to fault isolation, and recovery to error recovery. The mapping is an analogy that guided the decomposition and is not a derivation.

Anomaly and intrusion detection. Denning (1987) proposed an intrusion-detection model in which behaviour of subjects is profiled from audit records and statistically abnormal behaviour is flagged. Axelsson (2000) surveys intrusion-detection systems and gives a taxonomy that distinguishes, among other things, detection by anomaly from detection by known patterns. Chandola et al. (2009) survey anomaly detection across domains, distinguish point, contextual, and collective anomalies, and identify persistent difficulties: defining what is normal, normal behaviour that evolves over time, scarcity of labelled anomalies, and adversaries who imitate normal behaviour. These difficulties are directly relevant: they imply that any detection layer will have non-zero error rates and that a design must state what it does when it errs.

Runtime verification and enforcement. Bartocci et al. (2018) introduce runtime verification as the checking of an execution trace against a formal property by a monitor, with attention to monitor synthesis and overhead. Alshiekh et al. (2018) propose *shielding*: a shield, built from a safety specification, monitors the actions of a learning agent and corrects those that would violate the specification. Both concern properties that can be written down in advance. Reachability-based safety analysis, for example as implemented in tools such as CORA (Althoff, 2015), computes sets of reachable states for dynamical systems; we cite it as an example of verification-style analysis for physical or hybrid systems, noting that it targets different dynamics than software agents and is not applied here.

Resource control. Control groups in the Linux kernel provide hierarchical accounting and limits for groups of processes (Menage, 2007), and the second version of the interface unifies the hierarchy for controllers such as CPU, memory, and I/O (Linux Kernel Documentation, 2024). These are mechanisms a containment layer could use on a host; they do not by themselves decide *when* to apply a restriction or coordinate a decision across hosts.

Agents. Jennings and Wooldridge (1998) and Wooldridge (2009) describe software agents as systems with autonomy, reactivity, proactiveness, and social ability. Autonomy is what makes runtime assurance different from monitoring a conventional program: the monitored entity pursues its own goals and may adapt to monitoring.

Decentralized building blocks. Decisions that need collective agreement fall under the classical Byzantine results (Lamport et al., 1982; Castro and Liskov, 1999); open identities raise the Sybil problem (Douceur, 2002); accountability after the fact depends on retained evidence (Weitzner et al., 2008). Telemetry, in this view, is part of that evidence.

DOI: https://doi.org/10.5281/zenodo.24078477

2.3 Orthogonality: Runtime Assurance and Verification

The claim to be tested is that the question 'is the agent behaving acceptably now?' is distinct from 'does this action satisfy its specification?'. The claim is not trivial, because runtime verification (Bartocci et al., 2018) *is* verification at runtime. Any monitoring requirement that can be written as a formal property over traces can be checked by a monitor derived from a specification, and that part belongs naturally with AetherProof. What remains is behaviour that has no specification, such as statistical deviation from a learned baseline, and the actuators (containment, halt, recovery), which are actions and not propositions. F2 tests whether the remainder is substantial.

2.4 Boundary with Governance

Governance decides whether an agent *may* act; runtime assurance decides whether an agent *is behaving* acceptably. The two meet at the halt: who may order an agent to stop is a governance question, and how the order is detected, attested, and enforced is a runtime question. If every runtime-specific element turned out to be either conventional monitoring or a governed action, runtime assurance would reduce to governance plus conventional monitoring, which is F3. Because AetherGovern has not itself passed its observable-benefit test, the boundary is drawn against a hypothesis, which weakens any conclusion from F3 in either direction.

2.5 Boundary with Operational Verification

AetherFlow's operational verification evaluates integrated changes after merge and remediates failures. Runtime assurance evaluates the *process* that produces changes, whether or not it is currently producing any. The boundary is clear in intent: one object is the integrated codebase, the other is a running agent. It could blur in practice if operational verification were extended to include agent-level telemetry, in which case the runtime candidate would be a bounded extension of AetherFlow, which is the case F1 considers.

2.6 The Falsification-First Methodology

The methodology applies the principle that a hypothesis deserves attention to the extent that it makes claims that could be shown false. Concretely, the failure conditions are stated first, and the evidence sought is evidence that *would show failure*. Evidence that merely agrees with the candidate is treated as weak, because a design described by its proponent can always be made to agree with itself. The method is the same as in the governance investigation; only the candidate and the boundaries differ.

2.7 Research Gap

The literature supplies mature techniques for resource control, anomaly detection, and runtime verification in centralized or single-host settings, and the Aether preprints supply architecture for five other concerns. What it does not supply, as far as this review can establish, is an analysis of whether process-level monitoring, detection, containment, halt, and recovery form a distinct concern for *decentralized, autonomous, code-modifying agents* and of whether that concern can be handled without reintroducing a central authority or breaking the determinism that AetherMesh requires. We state this as a gap in the material reviewed, not as a claim about all existing work.

Limitations of this section. The review is selective and not systematic. The anomaly-detection and runtime-verification literatures are large, and only representative surveys are cited. Statements about the Aether records follow the author's own preprints, which no independent reader has verified.

3. The Falsification-First Methodology

3.1 The Problem with Prior Assumptions

A candidate fits a narrative when it answers the next question in a sequence. The Aether sequence asks how changes are stored, integrated, proven, synchronized, and authorized; 'how are running agents kept in check?' fits naturally as the sixth question. The fit is exactly what should make us cautious: *(observation)* that a candidate fits a narrative is not evidence that it fills a gap.

Three assumptions deserve explicit challenge. **First**, that the existing preprints do not already cover runtime concerns; Section 2.1 observed that fragments exist (resource policies for deltas in AetherMesh, revocation in AetherGovern, log-based replay in AetherGit). **Second**, that runtime assurance is distinct from verification and from governance; Sections 2.3 and 2.4 explain why neither distinction is obvious. **Third**, that runtime assurance can be decentralized; any halt that must propagate through an eventually consistent substrate is intrinsically delayed, and a design that ignores this would be unsound.

3.2 The Five Failure Conditions (F1–F5)

Each failure condition is stated so that its *satisfaction* shows the candidate fails. For each we give the claim under test, what would falsify it, and the admissible evidence. Appendix B gives full detail.

ID	Claim under test	Falsified if	Admissible evidence
F1 Non-redundancy	Some runtime requirement among RR1–RR6 is not met by the existing dimensions' specified mechanisms plus a bounded, localized extension of one of them	Every requirement is met by existing mechanisms or by a bounded extension of a single dimension	Mapping of requirements to specifications; extension-size analysis
F2 Orthogonality to verification	Runtime-assurance decisions cannot all be encoded as verified propositions without loss	Every runtime decision, including detection and halt, can be encoded as a property checked by a monitor derived from specifications	Encoding attempt; analysis of what remains unspecified
F3 Boundary with governance	Runtime assurance is distinct from governance plus conventional monitoring	Every runtime-specific element is either a governed action or conventional monitoring	Element-by-element classification
F4 Decentralizability	Runtime assurance can be specified without a central authority and without breaking AetherMesh's determinism requirement (A6)	Any design preserving the required properties needs a central authority or strong consistency for a core operation	Formal argument; counterexample search
F5 Observable benefit	Some scenario classes exist in which the family with runtime assurance outperforms the family without it on a pre-specified metric	No scenario class can be constructed in which the best existing baseline fails and runtime assurance succeeds	Pre-registered simulation or deployment study (not yet done)

The requirement list used by F1. We define six candidate requirements. The list is itself a hypothesis, revisited in Appendix B.

ID	Requirement (definition)
RR1	Telemetry: per-agent resource and action telemetry, attributable to an agent identity and collected outside the agent's own control.
RR2	Detection: identification of deviations from an agent's expected behaviour, with a stated trade-off between false alarms and missed anomalies.
RR3	Containment: graduated, reversible restriction of an agent's resources and access, preserving forensic evidence.
RR4	Halt: stopping an agent by a verifiable, authorized order, including when the agent's host is uncooperative.

ID	Requirement (definition)
RR5	Recovery: reconciling state after containment or halt, replaying only actions judged safe, and returning to a known-good state.
RR6	Partition behaviour: containment and halt remain usable at a disconnected node, with a stated safety consequence.

3.3 The Decision Rule

The rule is stated before any outcome is known. **Not falsified is a weaker status than confirmed**, and 'not falsified' is never promoted to 'confirmed' on the basis of argument. We use *single*, *dual*, *triple*, and *quadruple* structure to mean, in our reading, developing one, two, three, or four capabilities as new work; the supplied brief names three candidates (orchestration, governance, runtime assurance) but allows four, so we leave open the possibility of a fourth, as yet unnamed, candidate.

Outcome	Conclusion	Consequence for project structure
F1 falsified	Requirements are already met or are a bounded extension of one dimension	No standalone project; fold the extension into that dimension
F2 falsified	Runtime assurance is a special case of verification	Fold into AetherProof
F3 falsified	Runtime assurance is governance plus conventional monitoring	Merge with the governance candidate; at most one project covers both
F4 falsified	Cannot be decentralized	Incompatible with the family's premises; do not pursue as a dimension
F5 falsified	No observable benefit	Do not pursue; retain the analysis as a negative result
None falsified, F5 untested (current state)	Not falsified; not confirmed	Undetermined; proceed to comparison and a pre-registered evaluation
None falsified, F5 supported by evidence	Supported in the scenarios tested	A standalone project becomes defensible; structure decided by comparison

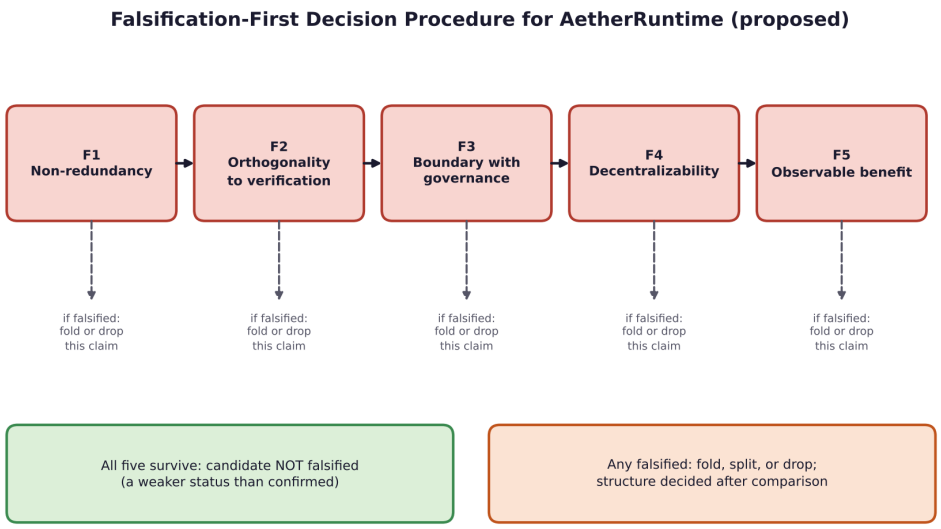


Figure 2 (Fig8_Falsification_Procedure.png; supplementary). The decision procedure. Surviving all five conditions yields 'not falsified', a weaker status than 'confirmed'.

3.4 Why This Methodology Matters

The methodology guards against narrative momentum, makes the paper's conclusion informative whichever way it points (a result that runtime assurance folds into AetherFlow or AetherProof would be as useful as one that it stands alone), and allocates effort honestly: the cheapest ways to be wrong (redundancy, reducibility) are examined first, the costly one (evaluation) last.

It also has limits. The same author proposes the candidate and the criteria, so the criteria may be too easy to survive; we mitigate this by stating the specific evidence that would falsify each and by naming residual weaknesses. An independent reviewer is a stronger check than any self-imposed criterion, and we invite one (Section 8.4).

Limitations of this section. The five conditions are one reasonable set, not an exhaustive or optimal one. The requirement list RR1–RR6 is a hypothesis; a different list could change F1's outcome.

4. The Three Candidate Capabilities

Runtime assurance is not the only possible next dimension. We describe three candidate capabilities so that the choice of subject is itself exposed to comparison. The descriptions define candidates, not designs; the names (AetherOrchestrate, AetherGovern, AetherRuntime) are working labels.

4.1 Orchestration (AetherOrchestrate)

AetherOrchestrate (candidate) *(definition).* A capability that decomposes goals into tasks, assigns tasks to agents, schedules and routes work, and tracks dependencies among tasks.

Gap hypothesized. The existing preprints do not say how work is divided among agents. **Decentralization question.** Orchestration is conventionally central; a decentralized version needs replicated task allocation, which raises agreement questions that gossip alone does not answer. **Principal risks.** The central version reintroduces a single point of failure; the decentralized version may need consensus and so fall under the impossibility results for asynchronous agreement. **Status.** Not assessed against F1–F5 in any Aether paper.

4.2 Governance (AetherGovern)

AetherGovern (candidate; Baladi, 2026e) *(definition).* A capability that determines which agents may perform which actions, under which policies, with what accountability, revocation, and dispute resolution.

Status. A preprint exists (Baladi, 2026e). It reports F1–F4 as not falsified by argument alone, with residual weaknesses, and F5 as untested; it is not a confirmed dimension. **Relevance here.** Revocation of authority is the network-level counterpart of a halt, and AetherGovern's exposure-window analysis is the model for P2. If AetherGovern is later folded into another dimension, the F3 boundary of this paper changes accordingly.

4.3 Runtime Assurance (AetherRuntime)

AetherRuntime (candidate) *(definition).* A capability that observes agents while they execute, detects deviations from expected behaviour, contains and if necessary halts agents, and recovers system state afterward.

Gap hypothesized. Verification and authorization do not constrain an agent between changes or within its permissions. **Decentralization question.** Monitoring is local, but halting an agent hosted elsewhere requires a collective decision that must be communicated and enforced. **Principal risks.** Overlap with

operational verification, with runtime verification, and with governance; false positives that halt legitimate agents; evasion by adaptive agents; and a new attack surface in which forged anomalies are used to halt honest agents.

4.4 Comparison of the Three Candidates

Criterion	Orchestration	Governance	Runtime assurance
Central question	What should agents do next?	May this agent act, under which policy?	Is this agent behaving acceptably now?
Gap hypothesized	Division of work	Authority, accountability, revocation	Behaviour during execution
Closest existing material	AetherFlow	AetherMesh key registry; AetherProof roles	AetherFlow operational verification; OS resource control
Redundancy risk	Medium	Medium	High: overlaps verification, governance, and OS facilities
Decentralization difficulty	High: agreement on assignment	Medium–high: revocation lag; trust anchors	Medium–high: collective halt; Sybil and halt-as-attack
Needs from the others	Authority to assign	Evidence of misbehaviour (telemetry)	Authority to halt; log for recovery
Written investigation	None	Baladi (2026e)	This paper
F1–F5 status	Not assessed	F1–F4 not falsified by argument; F5 untested	F1–F4 provisional (Section 5.8); F5 untested

Table 4.1. Comparison of candidate capabilities. Entries are the author's qualitative judgements, not measurements.

The table exposes a mutual dependence between the governance and runtime candidates: governance needs evidence of misbehaviour to decide on revocation, and runtime assurance needs authority to halt. This is an observation about dependency; it could equally be read as evidence that the two belong together (F3) or that they are complementary.

4.5 Which Capability Is the Strongest Candidate?

This paper does not rank the candidates. Runtime assurance is examined in depth because it has the highest redundancy risk of the three, which makes the redundancy question (F1, F2, F3) the most testable. A candidate that can be refuted cheaply is a good subject for a falsification-first investigation, whether or not it proves to be the best candidate.

> **Candidate statement** *(hypothesis).* AetherRuntime is a candidate for addressing the runtime-assurance gap, provided it passes the falsification criteria (F1–F5).

The three candidates will be compared after each has been assessed against analogous conditions (Q20); the final structure is undetermined.

> **Limitations of this section.** The orchestration description is brief and was not investigated, which biases the comparison. The governance entry rests on a preprint whose own conclusions are provisional. Table 4.1 is qualitative.

5. AetherRuntime: Architecture

This section specifies the hypothesized architecture. All of it is hypothesis: a specification-level description offered so that the failure conditions can be applied to something concrete. Nothing here has been implemented, and no real-world deployment has been evaluated.

5.1 The Five Layers

The architecture has five layers, each addressing one question. Their order in Figure 3 reflects conceptual dependency (each layer presupposes those below it), not an enforcement hierarchy. Figure 4 shows the corresponding lifecycle; classification of an anomaly (benign, suspicious, malicious) is part of the detection layer.



Figure 3 (Fig2_Runtime_Layers.png). The five layers of AetherRuntime (hypothesis). Ordering reflects conceptual dependency, not an enforcement hierarchy.

Layer	Question	Objects	Operations	Design requirement (hypothesis)
1 Monitoring	What is the agent doing now?	Telemetry streams; windows; signed summaries	collect, normalize, aggregate, export	Telemetry is collected outside the agent's control and attributable to an agent identity
2 Detection	Is this behaviour anomalous?	Baselines; detectors; classifications	compare, classify, report	Every detector states its error trade-off; verdicts are local and do not gate admission
3 Containment	How do we isolate the agent?	Containment levels C0–C4; forensic snapshot	restrict, release, snapshot	Graduated, reversible, host-local, evidence-preserving
4 Halt	How do we stop the agent?	Halt proposal; attestations; halt order	propose, attest, disseminate, enforce	Orders are signed, threshold-attested, and evaluated deterministically at admission
5 Recovery	How do we restore the system?	Action classification; quarantine marker; compensating record	classify, quarantine, replay, re-register	State is reconciled by appending records; nothing admitted is deleted

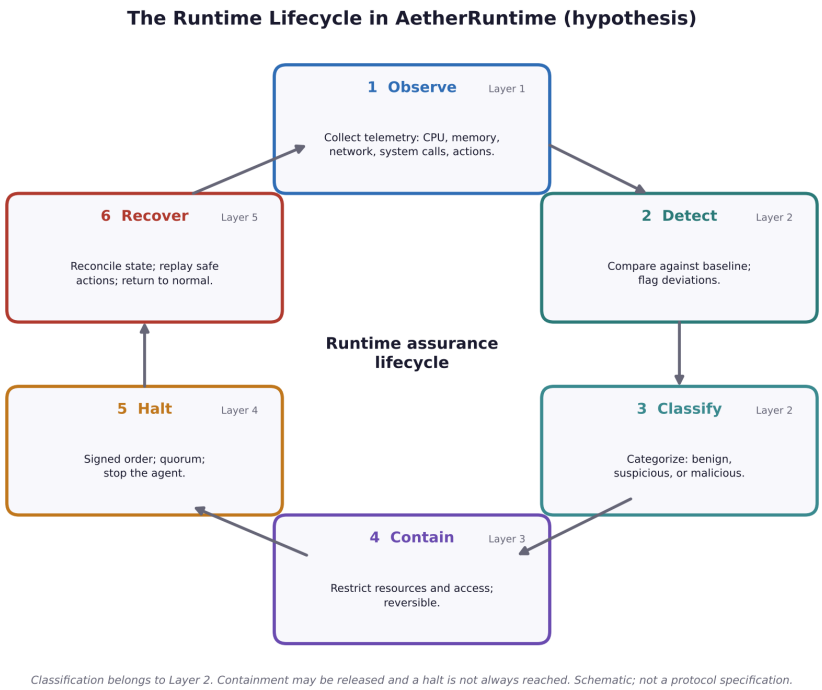


Figure 4 (Fig3_Runtime_Lifecycle.png). The runtime lifecycle (hypothesis): observe, detect, classify, contain, halt, recover. Containment may be released and a halt is not always reached. Schematic; not a protocol specification.

Limitations of this section. The decomposition into five layers is a choice, not a derivation. The dependability vocabulary of Avizienis et al. (2004) suggested it; other decompositions are possible.

5.2 Monitoring Pipeline

The monitoring layer produces the evidence on which every later layer depends. We describe the telemetry classes, the pipeline, and the integrity problem.

Class	Examples	Typical source (mechanism class)	Integrity concern
Resource use	CPU time, memory occupancy, I/O volume	Host resource accounting, for example control-group counters (Menage, 2007; Linux Kernel Documentation, 2024)	Accurate if the host kernel is trusted
Network	Bytes and connections per peer and per interval	Host network accounting	Agent may route through unmonitored paths if isolation is weak
System calls	Rates and sequences of calls	Host-level tracing	Overhead; tracing evasion
Action records	Deltas, merges, key operations	AetherGit-style log; AetherMesh deltas	Signed by the agent, so attributable but self-reported
Liveness	Heartbeats, progress markers	Agent or host	Can be spoofed by a compromised agent

Pipeline. Telemetry passes through five stages: *collection* from the host and from the agent's action stream; *normalization* to per-agent, per-interval quantities; *aggregation* over sliding windows; *local retention* for later review; and optional *export* as signed observation summaries that other nodes may consult (Section 5.5). Retention matters for accountability: in the sense of Weitzner et al. (2008), after-the-fact determination of what happened requires evidence that was kept.

The integrity problem. Telemetry is trustworthy only to the extent that its collector is. If the collector runs inside the agent's own trust boundary, a compromised agent can falsify it; if it runs on a compromised host, the host can. AetherProof's trusted-computing-base manifests record which tools a verification result

depends on; an analogous manifest for monitors would state what must be trusted for the telemetry to be believed (Q9). The architecture therefore assumes that the monitor and the agent are separated and that the host's monitor is not compromised, an assumption we name as a residual weakness of F4.

Limitations of this section. The list of telemetry classes is illustrative. The overhead of collection was not estimated, and no mechanism is specified for protecting telemetry from a compromised host.

5.3 Detection Layer

Detection compares windowed telemetry with expected behaviour and classifies deviations. We describe three detector families, the classification, and a resource-consumption metric.

Family	How it works	Strength	Weakness
Rule or threshold	Fixed limits on named quantities	Predictable; easy to audit	Brittle; needs hand-set limits; blind to novel patterns
Statistical baseline	Deviation from a profile of past behaviour (cf. Denning, 1987)	Adapts to each agent; finds unforeseen deviations	Baseline drift; cold start; false alarms (Chandola et al., 2009)
Learned classifier	Model trained on labelled or unlabelled behaviour	Can capture complex patterns	Needs data that may not exist; opaque; evadable by adaptive agents

Classification. A detector outputs one of three classes: *benign* (no action beyond logging), *suspicious* (containment may be applied locally and the observation may be exported), or *malicious* (a halt proposal may be issued). The classes are definitions for the architecture; the thresholds between them are policy and are unspecified.

Error trade-off. Any threshold trades false alarms against missed anomalies (Figure 5, panel b). Anomaly-detection surveys identify the difficulty of defining normal behaviour, normal behaviour that evolves, and adversaries who imitate normal behaviour (Chandola et al., 2009). We therefore state as a design requirement that each detector report its error trade-off, and as an *observation* that this paper cannot say where the trade-off should lie.

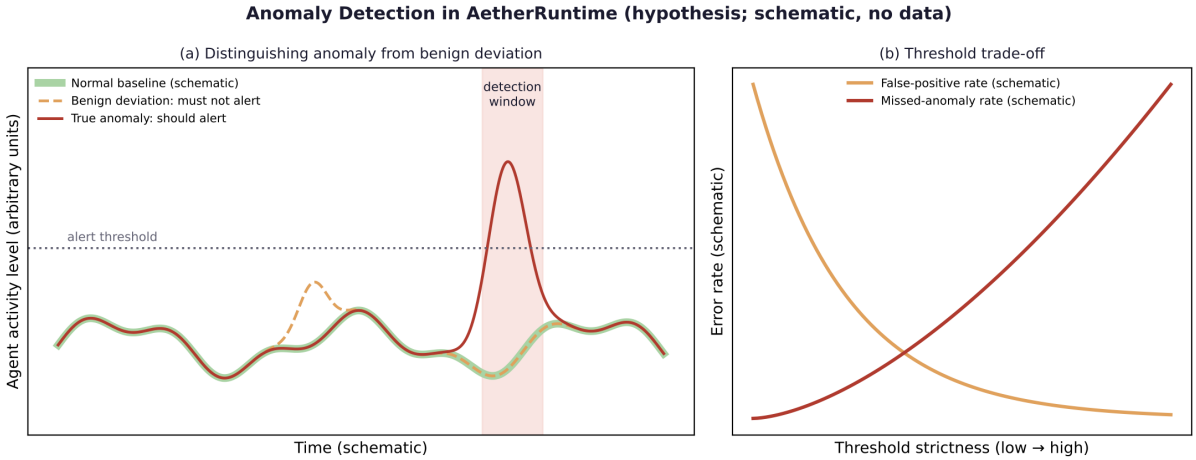


Figure 5 (Fig4_Anomaly_Detection.png). Anomaly detection (hypothesis; schematic, no data). (a) A benign deviation that must not alert and a true anomaly that should; the green band is the normal baseline, and the curves are drawn for illustration only. (b) The threshold trade-off between false positives and missed anomalies.

5.3.1 A resource-consumption metric

The draft formulation of the resource metric is, for agent i at time t:

$$\Omega_i(t) = \int_0^t \left(CPU_i(\tau) + Memory_i(\tau) + NetworkIO_i(\tau) \right) d\tau > \Theta_{max} \quad (1)$$

Equation (1) has four defects, which we state as observations about the draft form. **(a) Units.** CPU utilization is a dimensionless fraction (or a count of cores), memory is a quantity of bytes, and network I/O is a rate in bytes per second; their sum is not defined, and the integral has no single unit. **(b) Monotonicity.** Integrating from 0 makes Ω_i a non-decreasing function of t, so any agent that runs long enough exceeds Θ_{max} whatever its behaviour. **(c) Masking.** A sum lets an excess in one resource be hidden by slack in another. **(d) Calibration.** A single threshold ignores that agents and tasks have different budgets.

We propose the following windowed, normalized form as a hypothesis. For resources r in a set R, let $x_{i,r}(\tau)$ be the instantaneous usage of resource r, B_r a budget in the same unit as $x_{i,r}$, $w_r \geq 0$ weights summing to 1, and W a window length. Define:

$$\Omega_i^W(t) = (1/W) \int_{t-W}^{t} \Sigma_{r \in R}\, w_r \cdot x_{i,r}(\tau) / B_r \, d\tau \tag{2}$$

$$flag_i(t) = [\, \Omega_i^W(t) > \theta \,] \vee [\, \exists\, r : (1/W) \int_{t-W}^{t} x_{i,r}(\tau)/B_r \, d\tau > \theta_r \,] \tag{3}$$

Each ratio x/B is dimensionless, so Ω^W is dimensionless and is the weighted average budget utilization over the window; the window removes the monotonic growth; the per-resource clause in Equation (3) removes masking; and the budgets B_r carry the calibration, to be set per agent or per task. Memory is a level, so its windowed average is occupancy; for network I/O, x is a rate. The form is a candidate metric, not a validated one, and the choice of weights, budgets, window, and thresholds is open (Q17).

Hypothetical illustration (not a measurement). Let R = {CPU, memory, network} with equal weights 1/3 and budgets of 1 core, 2 GiB, and 10 MB/s. Suppose an agent used 2 cores, 1 GiB, and 5 MB/s throughout a window. The ratios are 2.0, 0.5, and 0.5, so $\Omega^W = (2.0 + 0.5 + 0.5)/3 = 1.0$. With $\theta = 1.5$ the aggregate clause does not fire, yet the CPU ratio of 2.0 exceeds a per-resource limit of $\theta_{CPU} = 1.5$, so the per-resource clause does. The numbers are invented to show masking and carry no empirical weight.

Limitations of this section. The metric is a proposal. Real agents have bursty usage, so the choice of W affects false alarms; adaptive agents can stay just under thresholds ('low and slow'). No detector was built or tested.

5.4 Containment Model

Containment restricts an agent without ending it. It is *host-local*: the node that hosts an agent can restrict that agent's resources and access without agreement from other nodes, because doing so does not alter the replicated state. Five levels are proposed, graduated so that the least restrictive adequate level can be chosen.

Level	Action	Mechanism class (illustrative)	Reversible	Evidence
C0	Observe only: raise logging detail	Telemetry configuration	Yes	Increases
C1	Throttle: lower CPU, memory, or I/O limits	Host resource controls (Menage, 2007)	Yes	Preserved
C2	Restrict network: allow only listed peers	Host network policy	Yes	Preserved
C3	Restrict filesystem: read-only or scoped	Host filesystem policy	Yes	Preserved
C4	Freeze: suspend the agent and take a forensic snapshot	Host process control	Yes (resume)	Snapshot taken first

Three principles govern containment. **Graduation**: prefer the least restrictive level that bounds the observed risk. **Reversibility**: every level can be released, so a false positive costs delay and not loss.

Evidence first: before a level that could destroy state (C4 and above), a snapshot is taken. Containment of an agent hosted elsewhere is not possible by this layer; the only network-level analogue, permanently refusing that agent's records, would break AetherMesh's convergence argument if done by local choice, and so belongs to the halt layer's governed path (Section 5.5).

Limitations of this section. Containment mechanisms depend on the host platform and were not surveyed. A containment level cannot protect against a compromised host.

5.5 Halt Protocol

A halt stops an agent. Two cases must be separated. A node can always stop an agent that *it hosts*, as a local emergency action, with no agreement from others. A halt that must bind *other nodes*, that is, one that makes them stop admitting an agent's records, needs a collective decision, because otherwise honest nodes would apply different local rules and diverge (AetherMesh's assumption A6). The protocol below concerns the second case.

Msg	Name	Content	Issued by	Verification at receiver
H1	HALT-PROPOSE	Agent id; level; evidence hashes; detector signature	Any detecting node	Signature valid; schema; evidence referenced
H2	HALT-ATTEST	Proposal hash; attester signature; own evidence assessment	Members of an attestation committee	Signature valid; member of committee
H3	HALT-ORDER	Proposal; at least t attestations	Any node holding t attestations	t distinct valid committee signatures
H4	REVOCATION	Revocation record for the agent's authority, citing the order	Governance layer, triggered by H3	Chain validity at cited frontier (Baladi, 2026e)
H5	HALT-ACK	Enforcement confirmation by the host	Hosting node	Signature valid

Committee and threshold. A committee of m members is sampled and an order requires t attestations, with $m/2 < t \leq 2m/3$. The lower bound prevents an order supported only by a Byzantine minority of fewer than one third; the upper bound keeps the protocol live when a third of the committee is silent. If each member is Byzantine independently with probability at most 1/3 (an approximation that is accurate when m is small relative to the network), the Chernoff–Hoeffding bound gives $P[\text{Byzantine count} \geq m/2] \leq \exp(-m \cdot D(1/2 \parallel 1/3))$, where $D(1/2 \parallel 1/3) = \tfrac{1}{2}\ln(3/2) + \tfrac{1}{2}\ln(3/4) \approx 0.0589$. The same bound limits the probability that Byzantine members can block an order for lack of honest attestations. Classical Byzantine agreement bounds (Lamport et al., 1982; Castro and Liskov, 1999) are the background for the one-third figure.

Committee size m	Upper bound on P[Byzantine count ≥ m/2]
25	2.29e-01
50	5.26e-02
100	2.77e-03
200	7.67e-06
400	5.88e-11

Table 5.1. Chernoff–Hoeffding upper bound exp(–0.0589·m) under independent sampling with Byzantine probability at most 1/3. Output of a formula under stated assumptions, not simulation results.

Enforcement. The order takes effect in two places. At the *host*, the agent is stopped (acknowledged by H5). At every *admitting node*, the order triggers a revocation record whose validity is evaluated deterministically at a cited frontier, so that the effect on the admitted set is the same at every honest node. If the host is uncooperative or compromised, only the second effect is available, and the agent's process may continue to run elsewhere but its records cease to be admitted.

Halt as an attack. A new attack arises: an adversary who forges anomalies, or controls a committee, can halt honest agents. The mitigations are design requirements: evidence hashes referenced in the proposal so that attesters assess evidence and do not merely vote; threshold attestation; rate limits on proposals per origin as delivery policies; and an escalation route for contested halts. We deliberately do not use per-node reputation as an input to admission, because reputation is local state and would violate the determinism requirement.

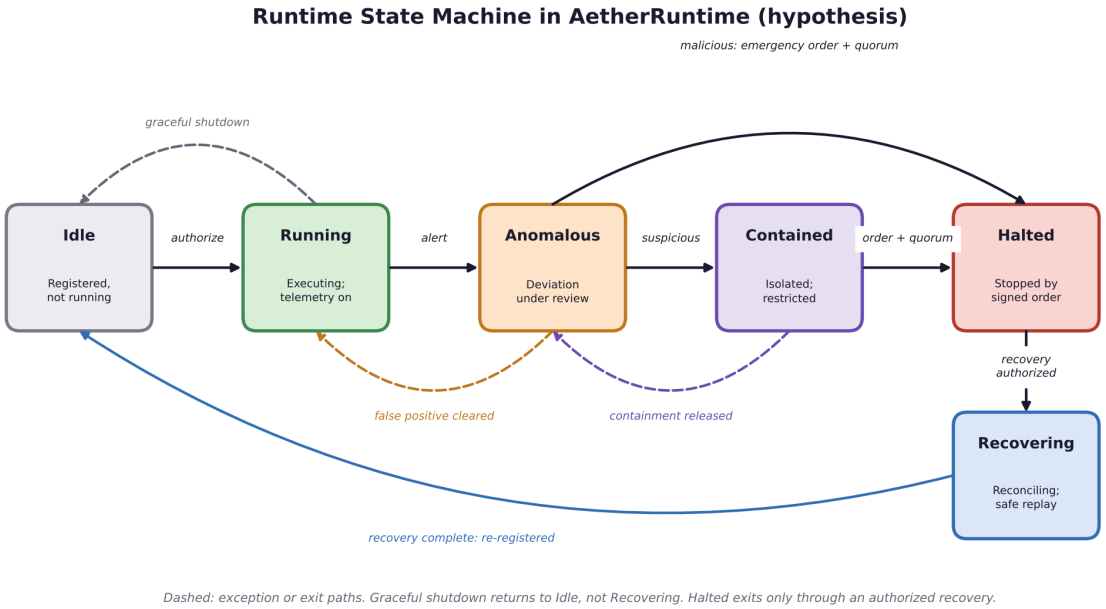


Figure 6 (Fig6_Runtime_State_Machine.png). The runtime state machine (hypothesis). Forward transitions run along the top row; dashed arcs show exits and releases. Graceful shutdown returns to Idle, not to Recovering. Halted exits only through an authorized recovery.

From → To	Guard (hypothesis)	Effect
Idle → Running	Authorized and registered (AetherGovern)	Telemetry begins; baseline selected
Running → Anomalous	Detector flags a deviation	Observation recorded; may be exported
Anomalous → Running	Review clears the alert	Alert closed; false positive recorded
Anomalous → Contained	Classified suspicious	Containment level chosen (C0–C4)
Anomalous → Halted	Classified malicious; halt order with t attestations	Host stops agent; revocation issued
Contained → Anomalous	Containment released	Restrictions lifted; review continues
Contained → Halted	Halt order with t attestations	As above
Running → Idle	Graceful shutdown	Agent deregistered
Halted → Recovering	Recovery authorized (governed action)	Reconciliation begins
Recovering → Idle	Recovery complete	Agent may be re-registered with a fresh baseline

Two structural properties of the state machine can be checked by inspection: every state has an exit, and the only way out of Halted is through an authorized recovery. Whether the machine is complete with respect to real failure modes is not claimed (Q18).

Limitations of this section. The committee-sampling bound assumes independence and a one-third Byzantine fraction, which are model assumptions. Committee selection, evidence assessment, and contested halts are unspecified. Table 5.1 is a formula, not an observation.

5.6 Recovery Procedure

After a halt (or a long containment), the system must return to a known-good state. Because AetherGit-style logs and AetherMesh state are append-only and inflationary, recovery cannot delete admitted records; it works by *adding* records that change how the state is interpreted. Five steps are proposed.

1. **Classify** the halted agent's actions since a suspected onset time into *safe* (independently verified or demonstrably independent of the anomaly), *suspect*, and *harmful*. The classification may use AetherProof evidence and AetherFlow checks.
2. **Quarantine** suspect and harmful actions by appending quarantine markers. The materialization function, which is a deterministic function of the delta set, treats marked deltas as excluded. The delta set still only grows, so the convergence argument is unaffected, and the materialized view changes deterministically at every node that holds the markers.
3. **Compensate** where an excluded action had side effects, by appending compensating changesets.
4. **Replay** safe actions that depended on excluded ones, after re-verification.
5. **Re-register** the agent, if at all, with a fresh baseline and an authorization decision (a governed action), and release containment.

This design mirrors the 'eventual validity' option identified in the governance investigation: later information changes the interpretation of earlier records. Quarantine markers are monotone additions, so they avoid rollback; the price is that the materialized state may change after the fact for readers who had already consumed it. Which actions are 'safe' is itself a hard judgement; independence of an action from an anomaly is not generally decidable from logs alone (Q12).

Limitations of this section. Recovery correctness is not defined formally. The classification step may rely on human judgement, and the interaction with consumers of the pre-quarantine state is unspecified.

5.7 Propositions 1 and 2

5.7.1 Determinism of local anomaly detection (P1)

The draft statement of the first proposition, in its mathematical form, is: with $S_i(t+1) = S_i(t) \sqcup \bigsqcup_{m \in M_i} m$, we have $\forall\, i, j$: $A_i(S_i) \wedge A_j(S_j) \Rightarrow (S_i \sqcup S_j) = (S_j \sqcup S_i)$. Two observations on this form. First, the conclusion holds for every pair of states by the commutativity of the join, whatever A_i and A_j are; it therefore does not express any property of anomaly detection. Second, the hypothesis that anomaly predicates are deterministic functions of *local* state is different from the requirement in AetherMesh's assumption A6, which asks that *admission* predicates be deterministic functions of a delta and its causal closure; a predicate that reads local telemetry, if used to admit or reject remote deltas, would violate A6. We therefore restate the proposition so that it addresses the real question: how can detection, which is inherently local, coexist with an admission rule that must not depend on local state?

Observation record *(definition).* A signed, schema-conformant record in which a node asserts that it observed a stated condition for a stated agent in a stated window, with evidence hashes. An observation record asserts *that the node observed* the condition, not that the condition holds.

Proposition 1 (P1: coexistence of local detection with admission determinism) *(stated with proof sketch; not machine-checked).* Suppose (a) AetherMesh admission predicates, including the signature, structural, schema, and compatibility checks and the causal-authority check of AetherGovern, are deterministic functions of a delta and its causal closure; (b) detection verdicts D_i, which may depend on node i's local telemetry history, affect only (i) containment actions on agents hosted at i and (ii) the *creation* of observation records, halt proposals, and attestations by i; and (c) a halt order affects

admission only through a revocation record evaluated at the frontier it cites. Then the set of deltas admitted by every honest node, once all created deltas have been delivered, is independent of the detection verdicts D_i at any node except through the set of deltas that were created, and the admission layer continues to satisfy assumption A6.

Proof sketch. By (a), the verdict on any delta d depends only on d and its causal closure; by (b), D_i does not appear in that function. The verdicts D_i influence the system in two ways: by changing which deltas are *created* (the set M* of AetherMesh's Theorem 1), and by changing local hosted-agent behaviour, which affects the replicated state only through further created deltas. AetherMesh's Theorem 1 quantifies over an arbitrary finite set M* of created deltas: for every such set, all honest nodes that receive M* converge on the join of the admitted subset, and by (a) the admitted subset is the same at every honest node. Hence different detection verdicts change M*, not the verdict function, and convergence holds for each. By (c), a halt order cannot act on admission except as a revocation record, whose evaluation is deterministic at its cited frontier (Proposition 1 of Baladi, 2026e). ∎ (sketch)

What the sketch does not show. It guarantees determinism and not usefulness: two nodes may legitimately create different observation records about the same agent, and the system holds both. It assumes that conditions (a) to (c) are respected by every predicate and every integration, which nothing enforces mechanically (Q11). It does not address the integrity of telemetry (Section 5.2). It inherits the status of AetherMesh's Theorem 1, which is itself a sketch, and of AetherGovern's Proposition 1.

5.7.2 Irreducible halt window (P2)

The draft statement of the second proposition decomposes the window as $\Delta t_{window} = t_{halt_enforced} - t_{detect} = D_{gossip}(N, f) + t_{queue} + t_{verify}$, and says that the window is bounded below by those terms. We state it as an inequality, add the attestation step that the protocol of Section 5.5 requires, and separate detection latency.

$$\Delta t_{window} = t_{enforced} - t_{detect} \geq t_{attest} + D_{gossip}(N, f; q) + t_{queue} + t_{verify} \quad (4)$$

Proposition 2 (P2: irreducible halt window) *(stated with proof sketch; not machine-checked).* In any decentralized halt protocol in which nodes act on locally held information and halt orders propagate by message passing, the time from detection to enforcement at a node is at least the time for the order to be formed and to reach that node, plus the time to queue and verify it there. For enforcement at a fraction q of nodes, $D_{gossip}(N, f; q)$ denotes the q-quantile of dissemination time to honest nodes when f nodes are Byzantine.

Proof sketch. Enforcement at node X is an event at X. The only information that tells X to enforce is the order, so the enforcement event is causally after the receipt event at X. Receipt is causally after the formation of the order, which is after t attestations have been produced, each after the proposal, which is after detection. Each causal step takes non-negative time; transmission takes at least the minimum path latency, verification at least the cost of checking t signatures. Any rule that enforced earlier would need X to act without the order, which is coordination-free action on information X does not have. Hence the inequality, and the window is non-zero. For enforcement at a fraction q of nodes, the last term is the q-quantile of dissemination time. ∎ (sketch)

Remarks. (i) Equality is a modelling decomposition that holds when the stages do not overlap; pipelining can make the total less than the sum of stage times but not less than the critical path. (ii) The *total* exposure of the system to a compromised agent is the detection latency plus the window; detection latency (Figure 7) is not part of Δt but may dominate it. (iii) For a compromised host that ignores the order, the window is defined for admitting nodes only. (iv) The proposition is an argument from causality, as in the governance investigation, and not a formal impossibility theorem.

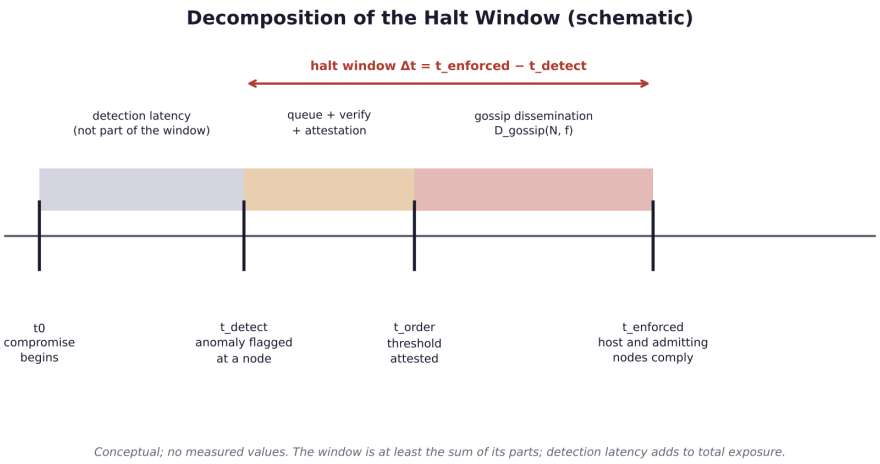


Figure 7 (Fig7_Halt_Window.png; supplementary). Decomposition of the halt window. Detection latency precedes the window; the window comprises attestation, queueing and verification, and dissemination. Conceptual; no measured values.

Hypothesis H1 (dependence on Byzantine fraction) *(hypothesis; not established).* If Byzantine nodes silently drop halt orders they receive, the effective gossip fanout is reduced from k to approximately k(1 – f/N), and the dissemination time grows accordingly.

Using the refined push estimate of AetherMesh, rounds ≈ ln n · (1/ln(1+k′) + 1/k′) with k′ = k(1 – f/N), n = 250, k = 5, and r = 10 rounds per second, the heuristic dissemination times in Table 5.2 follow. These are computed from a formula; they are not simulation output, and the formula itself coincided with a simulated value at only one network size in AetherMesh's evaluation.

Byzantine fraction f/N	Effective fanout k′	Rounds (heuristic)	Dissemination time, ms (heuristic)
0.00	5.00	4.19	419
0.10	4.50	4.47	447
0.20	4.00	4.81	481
0.33	3.33	5.42	542

Table 5.2. Heuristic dissemination time of a halt order to honest nodes (n = 250, k = 5, r = 10 Hz) under the silent-drop model of H1. Excludes attestation, queueing, and verification. Formula output, not data.

Limitations of this section. P1 and P2 remain proof sketches, not machine-checked. P2 is an argument from causality. H1 depends on an unverified estimate and on an adversary model (silent dropping) that is only one of several; an adversary could also delay selectively.

5.8 Provisional Assessment

We apply the failure conditions to the candidate as specified, using argument alone. The results record whether we found a way to falsify each condition, not whether none exists.

Condition	Attempted falsification	Result	Residual weakness
F1 Non-redundancy	Map RR1–RR6. RR1 (telemetry), RR2 (detection), RR3 (containment) and RR6 (partition behaviour) are not specified in any of the five preprints. RR4 overlaps AetherGovern's revocation at network level, but process-level stop is uncovered. RR5 is partly covered by the AetherGit log and AetherFlow remediation.	Not falsified for RR1–RR3, RR6; partial for RR4–RR5	A bounded extension of AetherFlow's operational verification might absorb RR1–RR2 (Q1); no review of the host-resource and intrusion-detection literature beyond surveys (Q2)

Condition	Attempted falsification	Result	Residual weakness
F2 Orthogonality to verification	Encode runtime decisions as properties. Properties with a specification (for example, never write outside scope; message rate within a stated budget) can be monitored by synthesized monitors and belong with AetherProof. Statistical detection without a specification, and the actuators, cannot be so encoded.	Not falsified in narrowed form; the specified-property subset folds into AetherProof	If statistical detection is recast as a specification with probabilistic guarantees, F2 weakens (Q3); shielding (Alshiekh et al., 2018) sits on the boundary (Q19)
F3 Boundary with governance	Classify each element. Telemetry, detection, and containment resemble conventional monitoring; authorizing a halt is a governed action; dissemination of a halt shares AetherGovern's revocation window. Runtime-specific residue: observation records, threshold attestation, quarantine markers, and the local/network split of enforcement.	Not falsified, weakly: the most fragile condition	If governance absorbs attestation and quarantine, the remainder is integration of conventional monitoring (Q6); AetherGovern is itself unconfirmed
F4 Decentralizability	P1 preserves determinism if detection verdicts do not gate admission; threshold attestation avoids a central authority; P2 gives the irreducible window. Search for a core operation needing strong consistency: none found, but timeliness is limited.	Not falsified, with explicit residuals	Window is irreducible; halt-as-attack; Sybil committees (Douceur, 2002); compromised hosts and telemetry integrity (Q8–Q10)
F5 Observable benefit	Not attempted; requires an experiment.	Untested	Everything depends on it (Section 7)

The overall status is *not falsified on F1–F4 by argument, with residuals, F3 weakest; F5 untested*. **Not falsified is a weaker status than confirmed.** It justifies the evaluation design of Section 7 and the comparison with the other candidates; it does not justify a standalone project.

> **Limitations of this section.** The assessment is made by the same author who proposed the candidate, using argument alone. Independent review, a formal encoding attempt for F2, and a classification of F3's residue by someone other than the author would be stronger tests.

6. AetherRuntime: Interfaces to the Aether Family

This section asks how the candidate would interact with each of the five existing preprints. Each interface is a hypothesis about what the candidate would consume or produce; none has been implemented, and no cross-component interaction has been tested.

6.1 Interface to AetherGit

AetherGit (Baladi, 2026a) supplies a signed, content-addressed, append-only log of agent actions. For AetherRuntime it is (i) a telemetry source of the 'action record' class (Section 5.2), (ii) the substrate on which quarantine markers and compensating changesets are appended (Section 5.6), and (iii) the evidence store referenced by observation records and halt proposals by hash. **Open issue:** the log's tamper-evidence protects records after they are written, but not their truthfulness; a compromised agent can write false actions (Q9).

6.2 Interface to AetherFlow

AetherFlow (Baladi, 2026b) addresses operational verification of agent workflows. The runtime layer would consume its pre-execution checks as a baseline (what the workflow was expected to do), and its remediation hooks could serve as the recovery step's compensation actions. **Open issue:** a bounded extension of AetherFlow might absorb monitoring and detection and so make the candidate redundant; this is the main F1 risk (Q1).

6.3 Interface to AetherProof

AetherProof (Baladi, 2026c) concerns verification and evidence. Properties that can be written as specifications can be monitored with runtime-verification techniques (Bartocci et al., 2018) and belong there; the runtime layer would supply the observed traces and the enforcement actuators. AetherProof's trusted-computing-base manifests are a template for stating what monitor correctness depends on. **Open issue:** the boundary between 'specified property monitored' and 'statistical anomaly detected' (Q3, Q19).

6.4 Interface to AetherMesh

AetherMesh (Baladi, 2026d) provides decentralized, convergent state propagation. The runtime layer relies on it for disseminating observation records, halt proposals, attestations, and orders, and must respect its admission-determinism assumption A6 (Proposition 1). **Open issue:** halt-order dissemination time depends on gossip behaviour that AetherMesh's evaluation validated only partially; Table 5.2 is heuristic.

6.5 Interface to AetherGovern

AetherGovern (Baladi, 2026e) provides causal-authority admission and revocation. A halt order that must bind other nodes is enforced as a revocation, and authorization to register, halt, or recover an agent is a governed action. **Open issue:** the revocation window is irreducible and shared with P2, and AetherGovern's own status is provisional; if governance absorbs attestation and quarantine, the runtime residue shrinks (F3, Q6).

Component	Runtime consumes	Runtime produces	Main open issue
AetherGit	Action records; append-only store	Quarantine markers; compensating records	Truthfulness of self-reported actions
AetherFlow	Expected-behaviour baselines; remediation hooks	Anomaly reports	Possible absorption of monitoring (F1)
AetherProof	Specified properties; TCB manifests	Observed traces; enforcement	Boundary with statistical detection (F2)
AetherMesh	Dissemination; deterministic admission	Observation records; halt messages	Determinism under local detection (P1)
AetherGovern	Authority chains; revocation	Halt proposals; recovery requests	Shared revocation window; boundary (F3)

Table 6.1. Interface summary. All entries are hypothesized interfaces.

Limitations of this section. The interfaces were derived from the published descriptions of the preprints, not from running code. Interactions between components in failure conditions were not examined.

7. Evaluation Design

No evaluation is reported. This section specifies what an evaluation would measure, against what, and how, so that the candidate can be refuted or retained on evidence. It is a plan, not a result.

7.1 Metrics M1–M9

ID	Metric	Definition (planned)	Relates to
M1	Detection latency	Time from anomaly onset to detector flag	P2, F5
M2	False-positive rate	Benign windows flagged / benign windows	F5
M3	False-negative rate	Injected anomalies missed / injected anomalies	F5
M4	Containment time	Flag to restriction in effect at host	F5
M5	Halt window	Proposal to enforcement at quantile q of admitting nodes (Eq. 4)	P2, F4
M6	Blast radius	Actions by the faulty agent admitted before enforcement	F5
M7	Recovery time	Halt to restored consistent view	F5
M8	Monitoring overhead	Extra CPU, memory, and bandwidth versus unmonitored	F4, F5
M9	Halt-attack resistance	Honest agents halted under a forged-anomaly adversary	F4

7.2 Baselines B1–B5

ID	Baseline	Purpose
B1	No runtime layer (existing five components only)	Tests F5: is there any benefit?
B2	Static resource limits and rate caps only	Tests whether detection adds anything over limits
B3	Governance revocation by manual operator decision	Tests whether automated halt beats human response
B4	AetherFlow with a bounded monitoring extension	Tests F1: absorption by an existing component
B5	Conventional centralized monitor (not decentralized)	Bounds the cost of decentralization (F4)

7.3 Methodology

1. **Testbed.** A simulated multi-agent network with controllable size, Byzantine fraction, and latency, replaying realistic action streams where available.
2. **Fault injection.** Inject resource exhaustion, runaway loops, data corruption, low-and-slow abuse, and colluding agents with known onset times; label ground truth by construction.
3. **Pre-registration.** Fix metrics, thresholds, and the decision rule of Section 3 before running, so that results cannot be selected after the fact.
4. **Replication.** Repeat across seeds and network sizes and report distributions and intervals, not single runs.
5. **Adversarial review.** Invite an independent reviewer to design evasions against the detectors and attacks against the halt path.

The decision rule is that of Section 3: the candidate is retained only if it is not falsified on F1–F4 and shows a benefit over B1 to B4 on M1 to M9 that is large enough to justify its overhead; the threshold is to be fixed in advance (Q16).

7.4 Threats to Validity

- **Construct:** simulated anomalies may not resemble real ones; ground-truth labels by construction favour detectors tuned to the injector.
- **Internal:** thresholds tuned on the test faults will overstate detection performance.
- **External:** results on a simulated network may not transfer to real deployments; no real-world deployment has been evaluated.

- **Author bias:** the same author proposes the candidate, the faults, and the baselines; independent replication is needed.
- **Model assumptions:** independence of committee sampling, a one-third Byzantine bound, and honest clocks are assumptions that may not hold.

7.5 Open Questions Q1–Q20

Q1	Can a bounded extension of AetherFlow absorb monitoring and detection (F1)?
Q2	What does the host-resource and intrusion-detection literature already provide that makes RR1–RR3 solved problems?
Q3	If statistical detection is recast as a specification with probabilistic guarantees, does F2 still hold?
Q4	Which baseline profile (per agent, per task, per role) is appropriate, and how is cold start handled?
Q5	How should detector drift be handled without letting an adversary retrain the baseline?
Q6	If governance absorbs attestation and quarantine, what runtime residue remains (F3)?
Q7	What attestation committee design (selection, rotation, size) balances liveness and capture?
Q8	How are Sybil committees prevented without a central identity authority (Douceur, 2002)?
Q9	What must be trusted for telemetry to be believed, and can a monitor TCB manifest be stated?
Q10	Can a compromised host be detected from the network side?
Q11	How can conditions (a) to (c) of P1 be enforced mechanically on new predicates and integrations?
Q12	When is an action independent of an anomaly, so it may be classed as safe in recovery?
Q13	What is the right policy for contested halts and for appeals?
Q14	How should halts interact with in-flight multi-agent transactions?
Q15	How should network partitions be handled, given that halt orders cannot cross them?
Q16	What effect size justifies the added overhead, and who fixes the threshold?
Q17	What weights, budgets, window, and thresholds should the metric of Eq. (2)-(3) use?
Q18	Is the state machine of Figure 6 complete with respect to real failure modes?
Q19	Where is the boundary between shielding (Alshiekh et al., 2018) and containment?
Q20	How do the runtime, orchestration, and governance candidates compare, and what is the final structure (single, dual, triple, or quadruple)?

Limitations of this section. The question list reflects what the author could identify; it is neither exhaustive nor ordered by importance.

8. Conclusion and Future Work

8.1 Summary

We examined whether a runtime-assurance layer, covering monitoring, detection, containment, halt, and recovery, could be a distinct component of the Aether family. We stated five failure conditions, specified a hypothesized five-layer architecture, restated two propositions as proof sketches, and assessed the candidate by argument. The provisional outcome is that the candidate is not falsified on F1 to F4, with the boundary with governance (F3) the weakest, and that F5 is untested. This is a candidate, not a result.

8.2 What the Evidence Supports

- The five existing preprints, as described, do not specify telemetry, detection, process-level containment, or partition behaviour (Observations 1 to 5).

- Local detection can coexist with deterministic admission if verdicts do not gate admission (P1, as a sketch).
- A non-zero halt window follows from causality in any decentralized design (P2, as a sketch).
- The draft resource metric had defects, and a windowed, normalized form avoids them (Section 5.3.1), though it is itself unvalidated.

8.3 What the Evidence Does Not Support

- That the architecture works, performs, or scales: no implementation, experiment, or measurement is reported.
- That the candidate is necessary: F1 and F3 could be refuted by a bounded extension or by absorption into governance.
- That the chosen detectors, thresholds, or committee sizes are right: they are unspecified.
- Anything about real-world deployment: no real-world deployment has been evaluated.
- That the numerical tables are empirical: Tables 5.1 and 5.2 are formula outputs under stated assumptions.

8.4 Future Work

In order: (1) independent review of the F1 to F4 assessment, especially F3 and the encoding attempt for F2; (2) assessment of the orchestration and governance candidates under analogous conditions (Q20); (3) a minimal prototype of the detection and containment path against B1 to B4, with pre-registered metrics; (4) simulation of the halt path, including a Byzantine and a halt-attack adversary, to test Table 5.2; (5) machine-checking of P1 and P2 if the candidate survives. Stopping after step 1 or 2 is a legitimate outcome.

8.5 Modest Conclusion

AetherRuntime is a candidate for addressing the runtime-assurance gap, and nothing more at present. It has survived a first, author-conducted attempt at refutation, which is a weak form of survival. **Not falsified is a weaker status than confirmed.** No real-world deployment has been evaluated. Propositions P1 and P2 remain proof sketches, not machine-checked. Whether the Aether family should include a runtime component, and in what structure, is undetermined.

DOI: https://doi.org/10.5281/zenodo.24078477

Appendix A. Notation and Definitions

Symbol	Meaning
$\Omega^W_i(t)$	Windowed, normalized resource utilization of agent i (Eq. 2)
$x_{i,r}$, B_r, w_r	Usage, budget, and weight for resource r
θ, θ_r, W	Aggregate threshold, per-resource threshold, window length
C0–C4	Containment levels (Section 5.4)
m, t	Committee size and attestation threshold
Δt_{window}	Detection-to-enforcement window excluding detection latency (Eq. 4)
$D_{gossip}(N, f; q)$	q-quantile dissemination time with f Byzantine of N nodes
$D(p\|q)$	Binary relative entropy, used in the Chernoff–Hoeffding bound
F1–F5	Failure conditions (Section 3)
RR1–RR6	Runtime requirements used for the redundancy mapping (Section 3)
M1–M9, B1–B5, Q1–Q20	Metrics, baselines, and open questions (Section 7)

Appendix B. Status Labels

Every statement in this paper carries one of the following statuses. **Definition**: a stipulation. **Observation**: a statement about the published preprints or about the draft material, checkable against them. **Hypothesis**: a claim not established. **Proof sketch**: an argument not machine-checked. **Formula output**: a number computed from a stated formula, not from an experiment.

Appendix C. Reconciliation Notes

The following differences between the draft specification and this paper are deliberate.

Item	Draft	This paper	Reason
P1	Join commutes for all states given predicates A_i, A_j	Coexistence of local detection with admission determinism	Draft conclusion holds trivially and conflicts with A6
P2	Window equals sum of terms	Lower bound including attestation	Stages may overlap; attestation was omitted
Ω	Integral of summed CPU, memory, and network	Windowed, normalized, per-resource clause	Units, monotonicity, masking
Menage	2006	2007 (unverified)	Year as understood by the author
Fig. 6	Graceful shutdown to Recovering; duplicate arrows	Redrawn: shutdown to Idle; malicious path added	Consistency
Fig. 5	Two boxes for a three-way title; unlabelled factor	Redrawn with three boxes; example labelled	Clarity

Appendix D. Reproducing the Numerical Tables

Table 5.1 is $\exp(-D \cdot m)$ with $D = \frac{1}{2}\ln(3/2) + \frac{1}{2}\ln(3/4) \approx 0.0589$. Table 5.2 uses $\text{rounds} = \ln n \cdot (1/\ln(1+k') + 1/k')$, $n = 250$, $k' = k(1 - f/N)$, $k = 5$, and time = rounds/r with $r = 10$ per second. Both are evaluated directly from the formulas; no simulation was run.

Figures. All figures are schematic and contain no measured data. Figure 1 (six dimensions) and Figures 3 to 6 follow the supplied drafts, redrawn where noted; Figures 2 and 7 are supplementary.

DOI: https://doi.org/10.5281/zenodo.24078477

References

Almeida, P. S., Shoker, A., and Baquero, C. (2018). Delta state replicated data types. *Journal of Parallel and Distributed Computing*, 111, 162–173.

Alshiekh, M., Bloem, R., Ehlers, R., Könighofer, B., Niekum, S., and Topcu, U. (2018). Safe reinforcement learning via shielding. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1).

Althoff, M. (2015). An introduction to CORA 2015. *Proceedings of the Workshop on Applied Verification for Continuous and Hybrid Systems*.

Avizienis, A., Laprie, J.-C., Randell, B., and Landwehr, C. (2004). Basic concepts and taxonomy of dependable and secure computing. *IEEE Transactions on Dependable and Secure Computing*, 1(1), 11–33.

Axelsson, S. (2000). The base-rate fallacy and the difficulty of intrusion detection. *ACM Transactions on Information and System Security*, 3(3), 186–205.

Baladi, S. (2026a). AetherGit. Zenodo. https://doi.org/10.5281/zenodo.23966896

Baladi, S. (2026b). AetherFlow. Zenodo. https://doi.org/10.5281/zenodo.23979129

Baladi, S. (2026c). AetherProof. Zenodo. https://doi.org/10.5281/zenodo.24013124

Baladi, S. (2026d). AetherMesh. Zenodo. https://doi.org/10.5281/zenodo.24038038

Baladi, S. (2026e). AetherGovern: A falsification-first investigation of governance in the Aether family. Zenodo. https://doi.org/10.5281/zenodo.24054524

Bartocci, E., Falcone, Y., Francalanza, A., and Reger, G. (2018). Introduction to runtime verification. In *Lectures on Runtime Verification*, LNCS 10457, Springer, 1–33.

Castro, M. and Liskov, B. (1999). Practical Byzantine fault tolerance. *Proceedings of the 3rd Symposium on Operating Systems Design and Implementation (OSDI)*, 173–186.

Chandola, V., Banerjee, A., and Kumar, V. (2009). Anomaly detection: A survey. *ACM Computing Surveys*, 41(3), Article 15.

Demers, A., Greene, D., Hauser, C., Irish, W., Larson, J., Shenker, S., Sturgis, H., Swinehart, D., and Terry, D. (1987). Epidemic algorithms for replicated database maintenance. *Proceedings of the 6th ACM Symposium on Principles of Distributed Computing (PODC)*, 1–12.

Denning, D. E. (1987). An intrusion-detection model. *IEEE Transactions on Software Engineering*, SE-13(2), 222–232.

Douceur, J. R. (2002). The Sybil attack. *Peer-to-Peer Systems (IPTPS)*, LNCS 2429, Springer, 251–260.

Jelasity, M., Montresor, A., and Babaoglu, O. (2005). Gossip-based aggregation in large dynamic networks. *ACM Transactions on Computer Systems*, 23(3), 219–252.

Jennings, N. R. and Wooldridge, M. (1998). Applications of intelligent agents. In *Agent Technology: Foundations, Applications, and Markets*, Springer, 3–28.

Karp, R., Schindelhauer, C., Shenker, S., and Vöcking, B. (2000). Randomized rumor spreading. *Proceedings of the 41st IEEE Symposium on Foundations of Computer Science (FOCS)*, 565–574.

Lamport, L., Shostak, R., and Pease, M. (1982). The Byzantine generals problem. *ACM Transactions on Programming Languages and Systems*, 4(3), 382–401.

Linux Kernel Documentation (2024). Control Group v2. https://docs.kernel.org/admin-guide/cgroup-v2.html

Menage, P. B. (2007). Adding generic process containers to the Linux kernel. *Proceedings of the Linux Symposium*, Vol. 2, 45–57. (Cited as 2006 in the draft list; year not independently verified.)

Shapiro, M., Preguiça, N., Baquero, C., and Zawirski, M. (2011). Conflict-free replicated data types. *Stabilization, Safety, and Security of Distributed Systems (SSS)*, LNCS 6976, Springer, 386–400.

Weitzner, D. J., Abelson, H., Berners-Lee, T., Feigenbaum, J., Hendler, J., and Sussman, G. J. (2008). Information accountability. *Communications of the ACM*, 51(6), 82–87.

Wooldridge, M. (2009). *An Introduction to MultiAgent Systems* (2nd ed.). John Wiley and Sons.

Research Status Statement

Architectural Hypothesis Under Investigation. AetherRuntime is a candidate, not a result. No implementation is claimed. No real-world deployment has been evaluated. Not falsified is a weaker status than confirmed. Propositions P1–P2 remain proof sketches, not machine-checked. The final project structure (single, dual, triple, or quadruple) is undetermined.

Author. Samir Baladi, Interdisciplinary AI Research, Ronin Institute / Rite of Renaissance. ORCID 0009-0003-8903-0029. gitdeeper@gmail.com. **DOI:** https://doi.org/10.5281/zenodo.24078477. Version 0.1.0, November 2026.